Bits, `is`, and Dictionaries

ISTA 130

Last Time

- Python list data type:
  - `split`, `join`
  - Mutability
  - Traversing
  - Methods

Bits

- Bit: short for “binary digit”. `0` or `1`. At the level of the CPU, computers work only with `0`’s and `1`’s. Everything we type in gets translated to bits.

Decimal vs. Binary

- Decimal numbers are base 10: single digits allowed are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. If we want to count higher than 9, we have to add another digit: 10, 11, 12, etc.
- Binary numbers are base 2: only possible digits are `0` and `1`. Mathematicians represent with subscript 2: $\mathtt{0110}_2$. In Python (and other computer languages), binary literals are prefixed by 0b: `0b0110` == 6_{10}.

Arithmetic in Binary

- `0 + 0 = 0`
- `0 + 1 = 1`
- `1 + 1 = ?`

Arithmetic in Binary

- `0 + 0 = 0`
- `0 + 1 = 1`
- `1 + 1 = 10`
- `1 + 1 + 1 = ?`

Arithmetic in Binary

- `0 + 0 = 0`
- `0 + 1 = 1`
- `1 + 1 = 10`
- `1 + 1 + 1 = 11`

Addition in Binary Works Like Decimal

-
- `  1 1 0 1`
- `+1 1 1 1`
-

Addition in Binary Works Like Decimal

```
      1
    1 1 0 1
  +1 1 1 1
  ---------
        0
```

Addition in Binary Works Like Decimal

```
    1 1
  1 1 0 1
+1 1 1 1
-------
  1 0 0
```

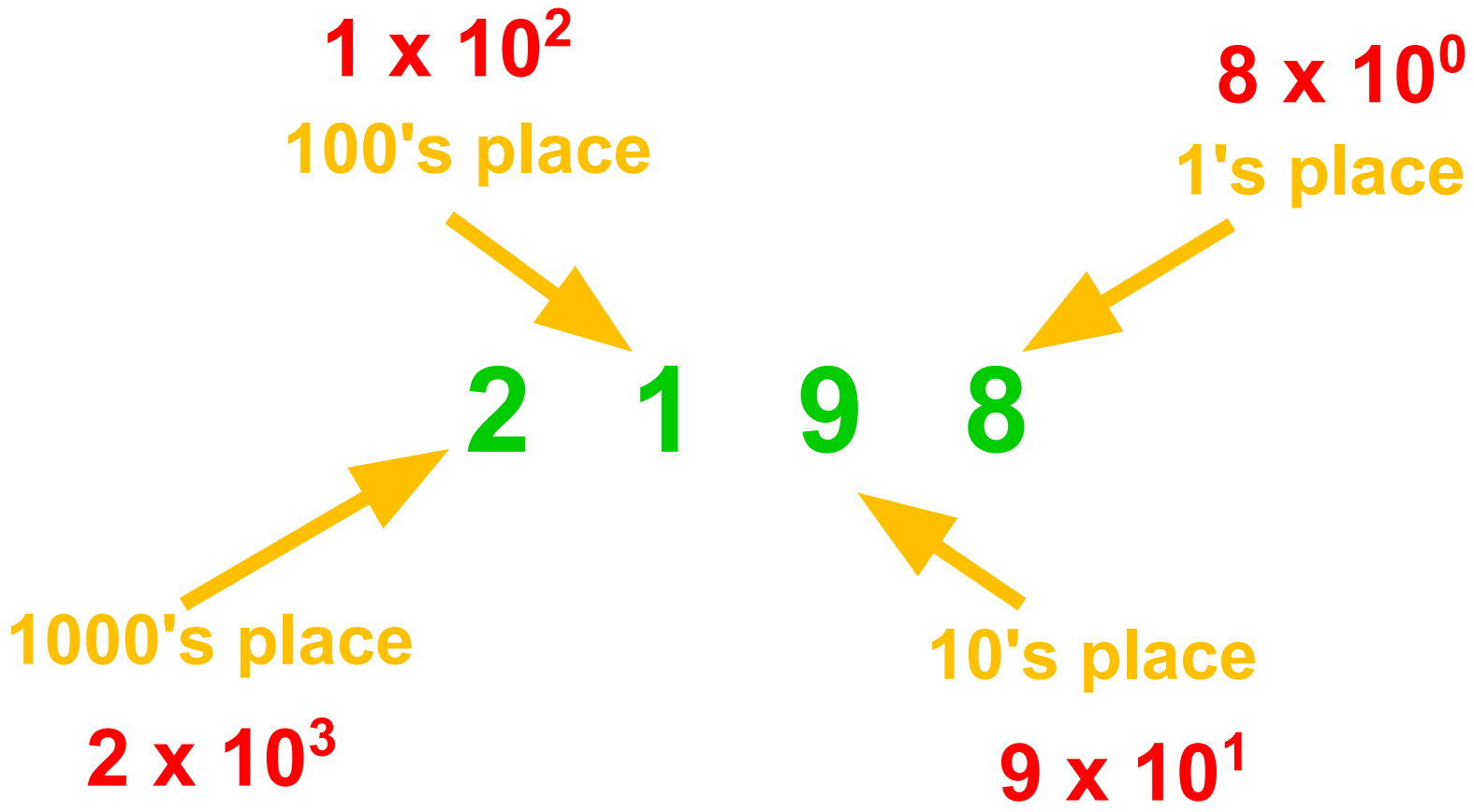
Addition in Binary Works Like Decimal

```
  1 1 1
  1 1 0 1
+1 1 1 1
-------
1 0 0
```

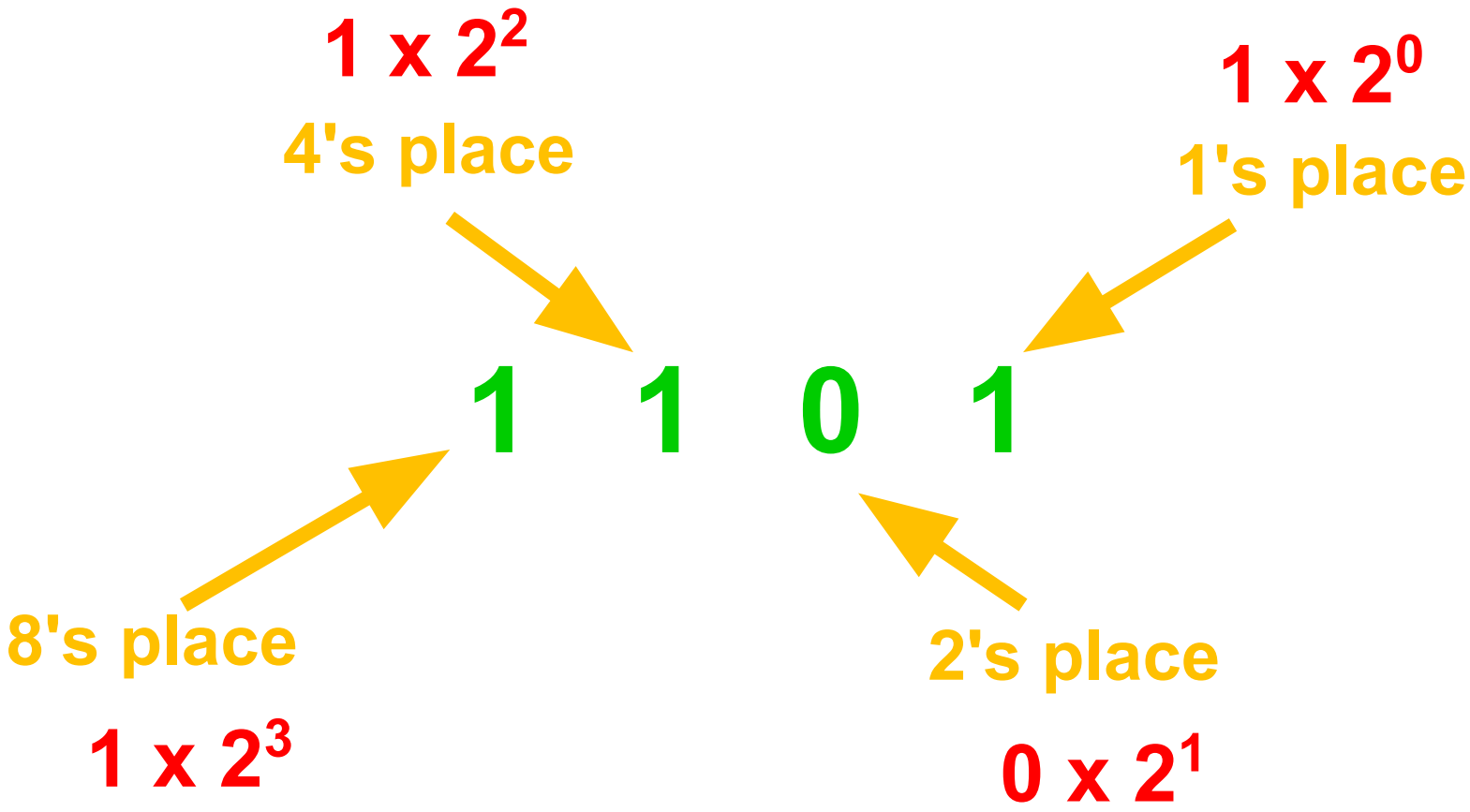
Addition in Binary Works Like Decimal

```
    1 1 1
  1 1 0 1
+ 1 1 1 1
-------
1 1 1 0 0
```

Decimal


1 x 10^2
100's place
8 x 10^0
1's place
2 1 9 8
1000's place
2 x 10^3
10's place
9 x 10^1

Binary



Converting Binary to Decimal

- **1010** =

  (1 * 2^{3}**) + (0** * 2^{2}**) + (1** * 2^{1}**) + (0** * 2^{0}**) =**

  (1 * 8**) + (0** * 4**) + (1** * 2**) + (0** * 1**) =**

  10

Decimal	Binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111


10:10


HOUR
MINUTE
11
00
Ajanta

The `is` Operator

- "Every object has an *identity*, a *type* and a *value*."[1]
  - everything we see in `Python` is an object (numbers, strings, turtles, lists, types, functions, modules, ...)
  - some objects have values that cannot change, they are *immutable* (e.g. strings)
  - some objects have values that can change, they are *mutable* (e.g. lists)
- The `==` operator compares two objects for *equality*
  - objects that have the same *value* are equal
- The **is** operator compares two objects for *identity*
  - objects that that are identical are the *same* object (i.e. there is only one object)

[1] For more details see http://docs.python.org/3.3/reference/datamodel.html

Comparing Lists with == vs. `is`

```
a = []
b = []

print(a == b) → True

# the values are equal:  both refer to empty lists

print(a is b) → False

# they are two distinct empty lists

a.append("linguini")
print(a) → ['linguini']
print(b) → []

print(a == b) → False
```

Comparing Lists with == vs. `is`

Continuing:

```
b.append("linguini")

print(a) → ['linguini']
print(b) → ['linguini']

print(a == b) → True

# they have the same value

print(a is b) → False

# but they are still two distinct lists
```

Comparing Lists with == vs. `is`

```
a = []
b = a

print(a == b) → True

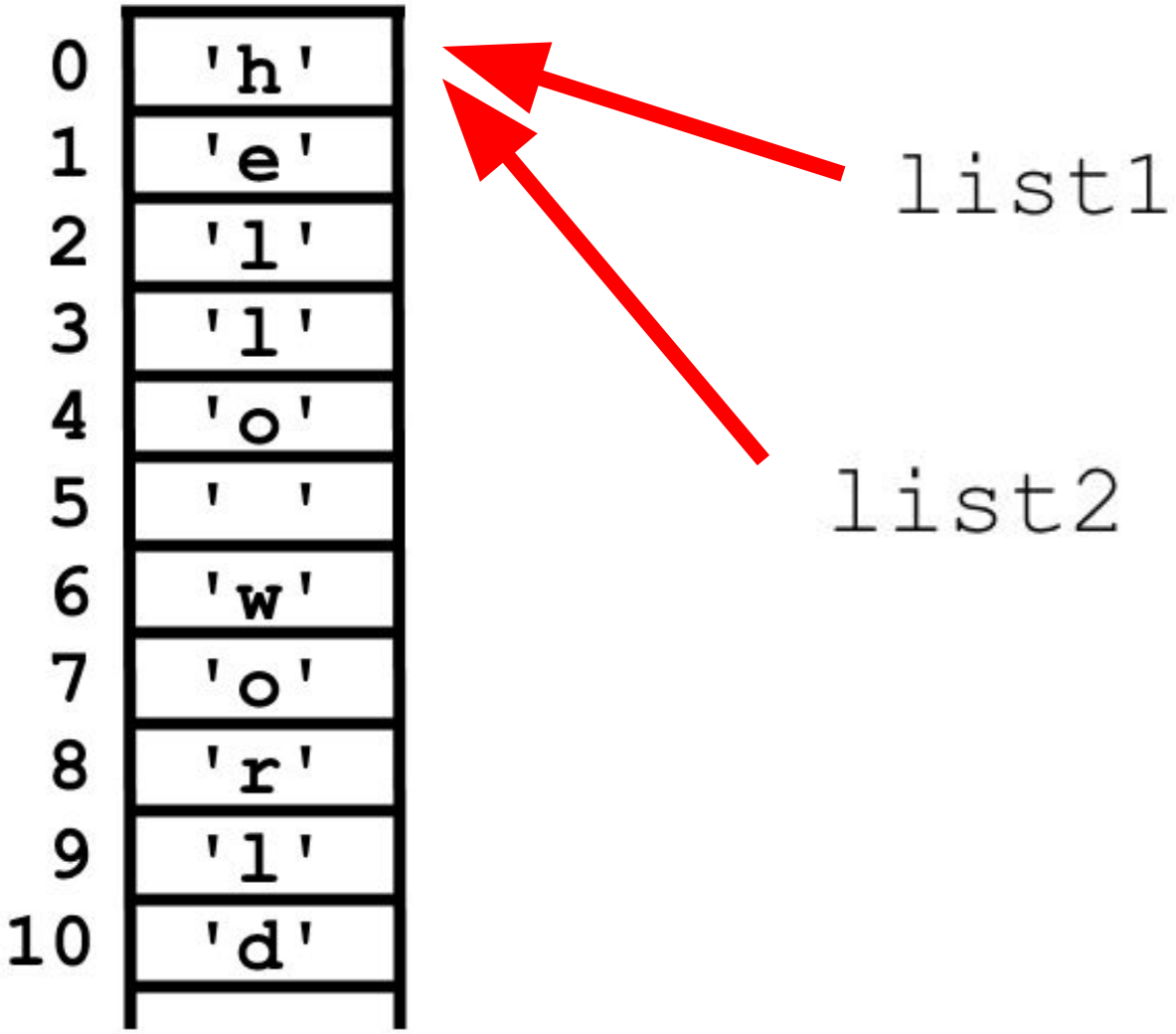
# values are equal:  both refer to an empty list

print(a is b) → True

# they are identical, both refer to the same list

a.append("linguini")
print(a) → ['linguini']
print(b) → ['linguini']
```

`list1 == list2 and list1 is list2`



list1 == list2 and list1 is not list2



DON'T Compare Numbers with `is`

- it is possible for two numbers to be equal but not identical[2]

```
a = 5
b = 5

print(a == b) → True # both refer to 5

print(a is b) → True # both refer to the same 5

a = 257
b = 257

print(a == b) → True # both refer to 257

print(a is b) → False # but different 257's!
```

[2] this is Python implementation dependent

DON'T Compare Strings with `is`

- it is possible for two strings to be equal but not identical

```
a = 'burp'
b = 'burp'

print(a, b) → burp burp

print(a == b) → True

print(a is b) → True

a = 'burp'
b = ''.join(['bu', 'rp'])

print(a, b) → burp burp

print(a == b) → True

print(a is b) → False
```

Python's `is` Operator

- **You will be tempted to use `is` – don't.**
- **At this level, the only time to use `is` is to check if a variable holds `None`, for example `if my_var is None:`**

Dictionaries

- A dictionary, as non-programmers know it, is a book which maps words to their meanings
  - a dictionary has a bunch of information in it, in the form of words paired with their definitions
  - given a word we can "look the word up" in a dictionary and find its definition
- A `dictionary` is also a built-in data structure in `Python`
  - a dictionary has a bunch of information in it, in the form of *keys* paired with their *values*
  - given a **key** we can "look the key up" in a dictionary and find its **value**

Dictionaries

Like a list, a dictionary allows us to store multiple values in a single container.

- With a list we access those values via index numbers
  - you can think of an index as the location of an element within a list
- With a dictionary we access the values via `keys`
  - you can think of a key as the location of an element within a dictionary

Dictionaries

- To create a brand new empty **list** object we did this:

```
a_list = []
```

- To create a new empty **dictionary** object we do this:

```
a_dictionary = {}
```

- Examine it in a shell:

```
print(a_dictionary) → {}
print(type(a_dictionary) → <class 'dict'>
```

Dictionaries

- We can also create a new empty **list** object like this:

```
a_list = list()
print(a_list) → []
```

- Similarly, we can create a new empty **dictionary** object:

```
some_dictionary = dict()
print(some_dictionary) → {}
```

Dictionaries

- To add an element to a dictionary we do this:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
```

- This maps the **key** "Manchego" to the **value** "Spain"
  - with dictionaries instead of an index we have a **key**
  - think of it as adding the word "Manchego" to the dictionary and giving it the definition "Spain"

```
print(cheeses) → {'Manchego':  'Spain'}
print(type(cheeses)) → <class 'dict'>
print(cheeses['Manchego']) → 'Spain'
print(type(cheeses['Manchego'])) →
   <class 'str'>
```

Dictionaries

- We can add as many *keys* to the dictionary as we like:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'
```

- Each key maps to a single value, together we refer to them as `key-value` pairs

```
print(cheeses) →
{'Leyden':  'Netherlands', 'Manchego':
'Spain', 'Cabrales':  'Spain'}

print(cheeses['Manchego']) → Spain
print(cheeses['Leyden']) → Netherlands
print(cheeses['Cabrales']) → Spain
```

Keys Must be Immutable

Only immutable type objects can be used as keys:

```
stuff = {}
stuff[1] = 'one'
stuff[2] = 'two'
stuff[''] = 'empty string'
stuff['apple'] = 'red'
stuff['ten'] = 10

numbers = [1, 2, 3]
stuff[numbers] = 'bunch of numbers'
TypeError: unhashable type: 'list'
```

Dictionaries

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Wensleydale'] = 'England'
cheeses['Cabrales'] = 'Spain'
```

- Notice when we print the entire dictionary:

```
print(cheeses) →
{'Leyden': 'Netherlands', 'Manchego':
'Spain', 'Wensleydale': 'England', 'Cabrales':
'Spain'}
```

Dictionaries

- The order of insertion is ~~not~~ maintained
  - Dictionaries are ~~*un*~~*ordered* collections of data

- We use a dictionary when we aren't interested in the order of insertion, we're just interested in the mappings from keys to values

As of Python 3.6, you can expect and rely on the order of key-value pairs being the same as the order you added the pairs to the dictionary!

Dictionaries being unordered is just part of Python's history and is relevant if you encounter an earlier version of Python.

Mutable

- Dictionaries are mutable:

```
print(cheeses) →
{'Leyden':  'Netherlands', 'Manchego':  'Spain',
'Cabrales':  'Spain'}

cheeses['Manchego'] = cheeses['Manchego'].upper()

print(cheeses) →
{'Leyden':  'Netherlands', 'Manchego':  'SPAIN',
'Cabrales':  'Spain'}
```

Dictionaries

- Look again at this code:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
```

- There was no element at key `'Manchego'` *before* executing this code.
  - With a list we would have gotten an error:

```
a = []
a[0] = 99
IndexError: list assignment index out of range
```

- With a dictionary this is how we add new key-value pairs

Dictionaries

- So the following code does one of two things:

```
cheeses['Cheddar'] = 'United Kingdom'
```

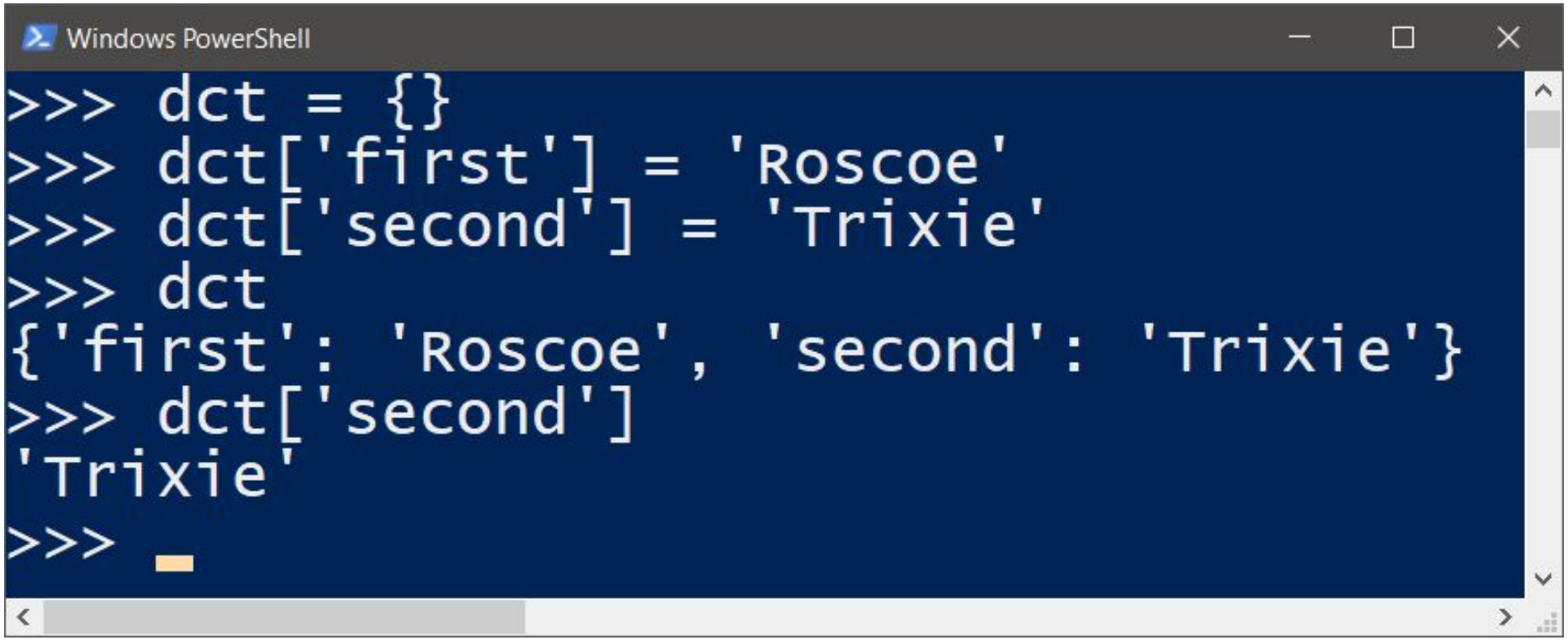
  1. If the **key** *was not* already present in the dictionary:
     - creates a new **key** and assigns the given **value** to it
  2. If the **key** *was* already present in the dictionary:
     - it assigns a new **value** to that key

- This is quite similar to the following code which also does one of two things:

```
cheddar = 'United Kingdom'
```

  1. If the variable `cheddar` *was not* already defined:
     - creates a new variable and assigns the given **value** to it
  2. If the variable `cheddar` *was* already defined:
     - assigns a new **value** to that variable

Dictionaries: Accessing a Value



Dictionaries: Key Error

- As with a list, attempting to access a non-existent element gives an error:

```
print(cheeses['Easy Cheese'])
```

```
Traceback (most recent call last):
File '/dictionary.py', line 15, in <module>
KeyError:  'Easy Cheese'
```

- We're trying to print the *value* of
  `cheeses['Easy Cheese']`
  - when Python tries to evaluate, it finds no value for the key `'Easy Cheese'` in the dictionary `cheeses`

List/Dictionary Comparison

A dictionary *maps* **keys** to **values**. You can think of a list as *mapping* **indices** to **values**.

critter_list

i	→	value
0	→	'African swallow'
1	→	'Badger'
2	→	'Cabbage'
3	→	'Donkey'
4	→	'Eagle'

food_dictionary

key	→	value
'ice cream'	→	'dessert'
'steak'	→	'dinner'
'soup'	→	'appetizer'
'coffee'	→	'life blood'
'fish'	→	'dinner'

- A list is ordered. We access its values by index.
  `print(critter_list[3])` → `Donkey`
- A dictionary is ~~not~~ ordered. We access its values by key.
  `print(food_dictionary['steak'])` → `dinner`

Dictionary Literals

- We've created *literal* lists with values already inside them:

```
my_list = [1, 2, 3]
```

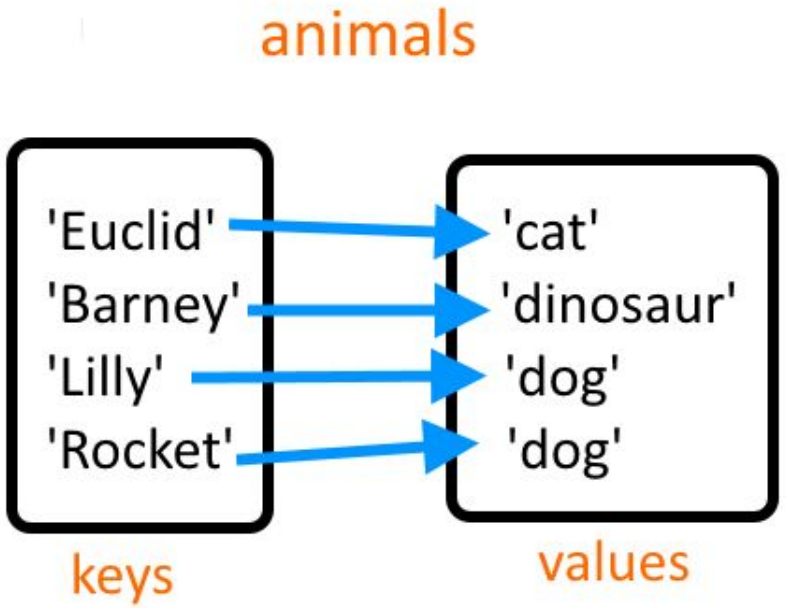
- We can create *literal* dictionaries with key-value pairs already inside them:
  - written the same way **print** displays dictionaries
  - separate keys from values with colons and separate key-value pairs with commas:

```
{key1: value1,  key2: value2, ...}
```

- For example, if asked to create a literal dictionary with food prices for a snack bar:

```
prices = {'ice cream':  1.99, 'chips':  0.99}
print(prices) →
{'chips':  0.99, 'ice cream':  1.99}
print(prices['chips']) → 0.99
```

Visualizing Dictionaries




```
animals = {}
animals['Rocket'] = 'dog'
animals['Euclid'] = 'cat'
animals['Lilly'] = 'dog'
animals['Barney'] = 'dinosaur'
```